

SAKAE SEIKI-iS 技術設計書

2026年8月7日 更新

目次

1 システム概要	4
1.1 実務フローの設計方針（重要）	4
2 全体アーキテクチャ	5
2.1 データ同期の仕組み	5
2.2 画面関係（作業票中心の構成）	5
3 ディレクトリ構成	6
3.1 新規追加ファイルの役割	6
4 認証	8
5 共有・同期レイヤー（_sakaeSync.js）	9
5.1 起動シーケンス（boot()）	9
5.2 保存フック（push）とpendingActions	9
5.3 sakaeSyncFlush()	9
5.4 新規タブを開く処理とポップアップ対策	10
5.5 初回同期（pull）	10
5.6 リアルタイム購読	11
6 データモデル	12
6.1 基準データと案件インデックス	12
6.2 TOPとの同期（recordsが古くならないようにする仕組み）	12
6.3 品番別データの構造（sakaeIS_buhinhyoMock_v1_<品番>）	12
6.3.1 共通項目（作業票と受注に関する連絡書で共有する項目）	13
6.3.2 連絡書固有項目	13
6.3.3 手配チェックシート固有項目	13
6.4 マイグレーション（互換処理）	13
6.5 localStorageキー一覧	14
7 画面別技術仕様	15
7.1 案件一覧（SAKAE SEIKI-iS.html）	15
7.2 作業票（作業票_モック.html）	15
7.3 個別日程表（個別日程表_モック.html）	15
7.4 作業開始印刷セット（作業開始印刷セット_モック.html）	16
7.5 作業票表紙（作業票表紙_モック.html）	16
7.6 手配チェックシート（手配チェックシート_モック.html）	16
7.7 カンバン（カンバン_モック.html）	16
7.8 受注に関する連絡書	17
7.9 作業票印刷（作業票印刷_モック.html）	17
7.10 NC日程表（NC日程表_モック.html）	17
7.11 実績入力（実績入力_モック.html）	17
7.12 工程別残品表（残品表_モック.html）	17
7.13 支給・納品一覧（支給納品一覧_モック.html）	17
7.14 各工程の日程表（各工程の日程表/）	18
8 既知の制限・注意点	19

8.1 動作確認済み仕様（実機Chromeで確認）	19
9 命名・コーディング規約（観察されたパターン）	20
10 デプロイ手順	21
11 用語・旧名称対応	22

対象読者: 開発に関わるエンジニア向け。仕様の背景と実装の勘所をまとめたもの。現状のフェーズ: 社内テスト運用版（本番の生産管理システムをそのまま置き換えるものではなく、実運用しながら検証中）。

1 システム概要

- 名称: SAKAE SEIKI-iS (榮製機株式会社 帳票一元管理システム)
- 目的: Excel・CSVでバラバラに管理していた受注～生産スケジュール～実績～残品管理を、1つの「品番」データを中心に連携させる
- 提供形態: ビルド工程・サーバーサイド言語を一切使わない、素のHTML/CSS/JSファイル群
- ホスティング: GitHub Pages (静的配信のみ)
- データ永続化・共有: Supabase (Postgres + Realtime + Auth)
- 対象ユーザー数: 7～8名 (Supabase Authに手動登録した社内アカウントのみ)

このシステムの最大の設計方針は「**各画面のロジックは一切変更せず、保存先だけをlocalStorage→Supabase経由の共有ストレージにすり替える**」という点。これは_sakaeSync.jsのコメントにも明記されている思想で、既存のモック（ローカルのみで動く単体HTML群）に極力手を入れずに複数人共有を実現するための設計判断。そのため各画面のビジネスロジックは今もlocalStorage.getItem/setItemで完結しており、Supabaseへの通信コードは各画面のHTML内には一切書かれていない。

1.1 実務フローの設計方針（重要）

このシステムは「**受注に関する連絡書から順番に書類を作るシステム**」ではない。実際の担当者の動きに合わせて、以下の方針で設計されている。

- **作業票を案件の中心画面として使用する。**部品・工程の入力、個別日程表の調整、各種帳票の出力は、すべて作業票を起点に行う
- 作業票・個別日程表・受注に関する連絡書は、**品番単位の共通データ (sakaeIS_buhinhyoMock_v1_<品番>) を直接参照する。**画面ごとに別々のデータを持ち、後から同期するという構造ではない
- 作業開始時に必要な3帳票（作業票表紙・手配チェックシート・カンバン）は、作業票上部の「作業開始印刷セット」からまとめて出力する
- **TOP画面（案件一覧）は受注書類の入力画面ではなく、案件を検索・選択して作業票へ進むためのポータルである。**新規登録の直後も、受注に関する連絡書ではなく作業票へ直接遷移する

「受注に関する連絡書」は業務上必要な正式書類として引き続き存在するが、システム操作の開始画面ではない。作業票・個別日程表を先に作成し、必要に応じて受注に関する連絡書を後から完成させる運用を前提にしている。

2 全体アーキテクチャ

2.1 データ同期の仕組み

[ブラウザ] --静的ファイル取得--> [GitHub Pages]

[ブラウザ内 JS]

各画面のロジック (getItem/setItem)

▼
Storage.prototype.setItem / removeItem をフック (_sakaeSync.js)

- ブラウザのlocalStorageにはそのまま書く (今まで通り)
- 500msデバウンス後、Supabaseのkv_storeテーブルへも書く

▼
[Supabase]

- Postgres: kv_store テーブル (key/value/updated_at/updated_by)
- Realtime: kv_storeの変更をpostgres_changesで全クライアントに配信
- Auth: メール+パスワードのユーザー認証 (サインアップ画面はなく、管理画面で手動発行)

要点:

- GitHub Pagesは静的ファイルを配るだけで、アプリケーションサーバーは存在しない
- 「サーバー」に相当する役割はすべてSupabaseが担っている
- Realtimeのpostgres_changes購読により、あるユーザーの変更が数百ms~数秒で他のユーザーのブラウザにも反映される

2.2 画面関係 (作業票中心の構成)

[案件一覧]

| productNoを渡す

▼
[作業票]

- [個別日程表]
- [受注に関する連絡書]
- [作業開始印刷セット]
 - [作業票表紙]
 - [手配チェックシート]
 - [カンバン]

上記の各画面は、それぞれ別々の案件データを持っているのではなく、すべて次の1つのキーを参照している。

sakaeIS_buhinhyoMock_v1_<品番>

作業票・個別日程表・受注に関する連絡書・作業開始印刷セット配下の3帳票は、この同一のJSONオブジェクトを直接読み書きする。画面間でデータをコピーして別々に保存する設計にはなっていない (例外として、TOP画面の案件カード一覧sakaeIS_records_v1はこのデータの検索用インデックスであり、詳しくはデータモデル章を参照)。

3 ディレクトリ構成

SAKAE SEIKI-iS/ ├── SAKAE SEIKI-iS.html ├── supabase_schema.sql 定) ├── GitHub公開_使い方.txt ├── Supabase共有テスト_使い方.txt ├── ドキュメント/ ├── システムファイル/ │ ├── _sakaeSupabaseConfig.js │ ├── _sakaeSync.js │ └── _workStartPrintCommon.js 共有する読み込み・整形処理 │ ├── _code39.js │ ├── ログイン.html │ ├── 作業票_モック.html │ ├── 作業票印刷_モック.html │ ├── 個別日程表_モック.html │ └── 作業開始印刷セット_モック.html □ │ ├── 作業票表紙_モック.html │ ├── 手配チェックシート_モック.html │ ├── カンバン_モック.html │ ├── NC日程表_モック.html │ ├── 実績入力_モック.html │ ├── 残品表_モック.html │ ├── 支給納品一覧_モック.html │ └── 各工程の日程表/ │ ├── _procGanttEngine.js │ ├── _procGanttEngine.css │ ├── index.html │ └── _カスタム日程表.html 成) │ └── MI日程表.html / LA日程表.html / ... 各工程コードごとの薄いHTML (エンジンをinitす るだけ) └── mikoshiya-seiki-is/ こちら) ├── .nojekyll るのを防止 │ ├── SAKAE SEIKI-iS.html │ ├── ドキュメント/ │ └── システムファイル/ ... 作業用フォルダ (GitHub非連携、ドラフト置き場) ... 案件一覧 (ポータル・TOP画面) ... Supabase側で実行するDDL (テーブル・RLS・Realtime設 ... 技術設計書・操作マニュアル (Markdown原本+PDF) ... Supabase接続情報 (URL・anon key・ON/OFFスイッチ) ... 共有レイヤー本体 (後述) ... 印刷3帳票 (表紙・手配チェックシート・カンバン) で ... バーコード (Code39) 関連ユーティリティ ... 認証ゲート (未ログイン時のリダイレクト先) ... 作業票 (案件の中心画面) ... 作業票の印刷用ビュー (部品単位でA4/A3出力) ... 個別日程表 ... 作業票表紙・手配チェックシート・カンバンへの入 ... 作業票表紙 (A5横・印刷専用ビュー) ... 手配チェックシート (A4横・入力可) ... カンバン (A4縦・上下2面印刷ビュー) ... NC日程表 ... 実績入力 ... 工程別残品表 ... 支給・納品一覧 ... NC以外の工程別ガントチャート (22工程分) ... 全工程共通の描画・操作エンジン ... 工程一覧 ... 未知の工程コード用フォールバック (?code=で動的生 成) ... GitHub Desktop連携リポジトリ (実際に公開されるのは ... Jekyllのデフォルト処理で"_"始まりファイルが除外され るのを防止 ... 上と同一構成 (コミット時にここへコピーして同期)

注意 (重要) : SAKAE SEIKI-iS/ 直下と mikoshiya-seiki-is/ (GitHub連携フォルダ) の中身は、実質同じファイルの二重管理になっている。編集は基本的に前者で行い、GitHub Desktopでコミット・プッシュする前に後者へ同一内容をコピーして反映する運用。両者に差分が出ると「直したはずなのに公開版に反映されていない」という事故につながるため、デプロイ前に必ず主要ファイルの内容一致を確認すること (diffで比較する運用にしている)。

GitHub PagesはデフォルトでJekyllが_始まりのファイル・フォルダをビルドから除外する。_sakaeSync.jsや_workStartなどが読み込めなくなる事故が過去に発生しているため、**リポジトリ直下に.nojekyll (空ファイル) が存在することが必須**。新しいリポジトリを作る際は最初に作成すること。

3.1 新規追加ファイルの役割

ファイル	役割
作業開始印刷セット_モック.html	作業票表紙・手配チェックシート・カンバンの3帳票への入口をまとめる画面。用紙サイズが異なるため、1回の印刷ジョブでは出さず、この画面から帳票ごとに順番に開いて印刷する
作業票表紙_モック.html	A5横の印刷専用ビュー。品番別データから基本情報・工程・着手/完成日・実働Hを自動反映する
手配チェックシート_モック.html	A4横の入力・保存・印刷画面。12項目の手配進捗（完了日・確認者）を入力・保存できる
カンバン_モック.html	A4縦の上下2面印刷ビュー。型番・品名を主表示、社内No・数量・客先・納期を補助表示する
_workStartPrintCommon.js	品番パラメータの取得、品番別データ（sakaeIS_buhinhyoMock_v1_<品番>）の読み込み・保存、手配チェックシート項目定義、営業日計算、実績時間集計など、印刷3帳票（+入口画面）で共有する処理をまとめたファイル

4 認証

- Supabase Authのメール+パスワード認証のみ（サインアップ画面は無し、Supabase管理画面から手動でユーザーを作成する運用）
- ログイン.htmlが唯一の非ゲート対象ページ（data-sakae-auth-page="login"をhtml要素に持たせ、_sakaeSync.jsが自分自身をガードしないようにしている。ここをガードすると無限リダイレクトになるため）
- ログイン成功後は?next=で渡された元のURLへリダイレクト
- 各画面は_sakaeSync.jsのロード時に自動でセッションチェックを行い、未ログインならログイン.html?next=<元のパス>へ強制遷移する（=各画面側に認証チェックのコードを書く必要はない）
- ログイン中のユーザーは、TOP画面右上の「パスワード変更」リンクから、window.sakaeSupabase.auth.updateUserを使って自分のパスワードを変更できる（サインアップ・パスワードリセットのメール送信フローは用意していない、最もシンプルな方式）

5 共有・同期レイヤー（_sakaeSync.js）

このファイルが本システムの技術的な核。全画面の<head>で_sakaeSupabaseConfig.jsの直後に読み込まれる。

5.1 起動シーケンス（boot()）

1. 画面全体を白いオーバーレイ（#sakaeAuthGate）で覆う（認証確認中のチラ見え防止）
2. Supabase JS SDK（CDN）を動的ロード
3. sb.auth.getSession() でセッション確認。無ければログイン画面へリダイレクトして終了
4. installLocalStorageHook() ... Storage.prototype.setItem/removeItemを上書き
5. initialSync() ... Supabase側の全キーをlocalStorageへ反映 + ローカルにしか無いキーをSupabaseへpush
6. subscribeRealtime() ... kv_storeの変更をリアルタイム購読開始
7. hideGate() ... オーバーレイを外して画面を見せる

各画面自身の初期描画コード（render()等）は、このboot()と並行してブラウザ内では実行されているが、オーバーレイに隠れているため利用者には見えない。hideGate()はinitialSync()完了後に呼ばれるため、利用者が実際に画面を目にする時点では最新データへの反映が済んでいる。

5.2 保存フック（push）とpendingActions

```
const pushTimers = {}; // キーごとのデバウンス用タイマー
const pendingActions = {}; // キーごとの「まだSupabaseに送っていない処理」本体
```

```
function schedulePush(key, value){
  if (pushTimers[key]) clearTimeout(pushTimers[key]);
  pendingActions[key] = () => pushKey(key, value);
  pushTimers[key] = setTimeout(() => pushKey(key, value), 500);
}
```

- isSyncKey(key) は key.indexOf('sakaeIS_') === 0 という単純なプレフィックス判定。アプリ側で新しいlocalStorageキーを追加する場合は、必ずsakaeIS_プレフィックスを付けること。付け忘れると同期対象から漏れる
- applyingRemoteUpdateフラグは、リモートから受信したデータをlocalStorageへ書き戻す最中にpushループが発生しないようにするためのガード
- デバウンスは500ms固定。編集直後にページ遷移すると、pushが発火する前にページが破棄され、その変更がSupabaseに反映されないまま消えるリスクがある。この対策がsakaeSyncFlush()（後述）

5.3 sakaeSyncFlush()

```
window.sakaeSyncFlush = async function(){
  const keys = Object.keys(pendingActions);
  if (!keys.length) return;
  const jobs = keys.map(k => {
    const fn = pendingActions[k];
    if (pushTimers[k]) { clearTimeout(pushTimers[k]); delete pushTimers[k]; }
    delete pendingActions[k];
    if (!fn) return Promise.resolve();
    try { return Promise.resolve(fn()).catch(e => console.warn('[sakaeSync] flush失敗:', k, e)); }
    catch (e) { console.warn('[sakaeSync] flush失敗:', k, e); return Promise.resolve(); }
  });
  await Promise.all(jobs);
};
```

要点（誤解しやすいので明記する）：

- _sakaeSync.js内に実装されている、ページ遷移前専用の関数
- 保留中の処理はpendingActions（キーごとに1関数）で管理されている

- 呼び出すと、保留中のpush・deleteタイマーを即座にキャンセルし、対応するpushKey()/deleteKey()（実際にSupabaseへupsert/deleteするPromiseを返す非同期関数）を**その場で実行する**
- Promise.all(jobs)で全キー分のSupabase処理が完了するまでawaitする。**単なる500ms待機ではない**。実際のSupabase通信（sb.from('kv_store').upsert(...)等）が終わるまでPromiseとして待機する
- 個々のキーの送信に失敗しても、他のキーの送信は継続する（.catchで握りつぶし、Promise.all全体は失敗させない）

使用箇所（入力直後のページ遷移による保存漏れを防ぐ）：

遷移元	遷移先	flush呼び出し
TOP（案件一覧）	作業票	あり（カードクリック・新規作成の両方）
作業票	個別日程表	あり
作業票	作業開始印刷セット	あり
作業票	受注に関する連絡書	あり
受注に関する連絡書	作業票	あり

いずれも、遷移直前にdocument.activeElement?.blur()でフォーカス中の入力（IME変換途中の文字など）を確定させ、save()でlocalStorageへ書き込んでからawait window.sakaeSyncFlush()を呼ぶ、という順序になっている。

5.4 新規タブを開く処理とポップアップ対策

非同期のflush処理を挟んだ後でwindow.open()を呼ぶと、多くのブラウザでポップアップブロックの対象になる。これは「ユーザー操作の直後」と見なされなくなるため。実機（GitHub Pages版）の動作確認で実際に発生した不具合であり、対策済みである。

現在の実装方針（作業票からの「個別日程表を開く」「現行の作業票印刷」「作業開始印刷セット」の各ボタンで採用）：

```
function openInNewTabAfterFlush(urlBuilder){
  const w = window.open('', '_blank'); // ① クリックした瞬間（awaitの前）に空タブを同期的に開く
  (async () => {
    document.activeElement?.blur();
    save();
    if (window.sakaeSyncFlush) { try { await window.sakaeSyncFlush(); } catch (e) {} } // ② flushを待つ
    const url = urlBuilder();
    if (w && !w.closed) w.location.href = url; // ③ 開いておいたタブへURLを設定する
  })();
}
```

この3ステップ方式により、以下の2つの制約を同時に回避している。

1. file://同士の遷移でwindow.open()がChromeに「unsafe attempt」としてブロックされることがある問題
2. awaitの後でwindow.open()を呼ぶとポップアップブロックされる問題

なお、受注に関する連絡書への遷移（作業票 → 受注に関する連絡書、その逆）は同一タブでのlocation.href遷移のため、上記のポップアップ問題は発生しない（新規タブを開かないため）。

5.5 初回同期 (pull)

```
async function initialSync(){
  const { data } = await sb.from('kv_store').select('key, value');
  data.forEach(row => applyRemoteRow(row)); // リモートの値でlocalStorageを上書き
  // ローカルにしか無いキー（＝Supabase未登録）はpush
}
```

applyRemoteRowは同一タブ内でも他画面のリアルタイム反映ロジックが動くよう、StorageEvent('storage', ...)を手動でdispatchする（ブラウザのstorageイベントは仕様上「他タブでの変更」でしか発火しないため、同一タブ内での反映にはこの手動発火が必要）。

5.6 リアルタイム購読

```
sb.channel('kv_store_changes')
  .on('postgres_changes', { event: '*', schema: 'public', table: 'kv_store' }, payload => {
    if (payload.eventType === 'DELETE') applyRemoteDelete(payload.old.key);
    else applyRemoteRow(payload.new);
  })
  .subscribe();
```

各画面側は、既存のwindow.addEventListener('storage', ...)ハンドラ（もともとローカルのマルチタブ対応用の実装されていたもの）をそのまま流用する形で、リモート変更への反応を実装している。画面側で新しくlocalStorageに保存するデータを増やす場合、対応するstorageイベントリスナーを書かないと、他ユーザーの変更がリアルタイムに反映されない（表示は次回ページ読み込み時まで古いまま）。作業票表紙・手配チェックシート・カンバン・作業開始印刷セットの4画面はいずれも、自分が表示している品番（sakaeIS_buhinhyoMock_v1_<品番>）の変更のみを購読している（実績ログsakaeIS_jissekiLog_v1は購読していない点に注意。詳細は実績入力章参照）。

6 データモデル

Supabase側はテーブル1本のみ。スキーマはsupabase_schema.sql参照。

```
create table kv_store (  
  key      text primary key,  
  value    jsonb not null,  
  updated_at timestampz not null default now(),  
  updated_by uuid references auth.users(id)  
);
```

RLSは「認証済みユーザーなら全件read/write可」のみ（キー単位・ユーザー単位のアクセス制御は無い＝全社員が全データを読み書きできる想定）。

6.1 基準データと案件インデックス

システム全体の基準データ（唯一の真実のソース）は以下である。

sakaeIS_buhinhyoMock_v1_<品番>

作業票・個別日程表・受注に関する連絡書・作業開始印刷セット配下の3帳票（作業票表紙・手配チェックシート・カンバン）は、すべてこのキーのJSONを直接読み書きする。

これに対して、以下はTOP（案件一覧）画面がカードを一覧表示するための**インデックスであり、基準データそのものではない**。

sakaeIS_records_v1

sakaeIS_records_v1は「品番・社内No・客先・型式・納期・ステータス」等、一覧表示や検索・CSV書き出しに必要な項目だけを持つ配列で、品番別データ（sakaeIS_buhinhyoMock_v1_<品番>）の内容が変わるたびに、必要な項目だけがコピー（同期）される。表示のたびに毎回全品番のデータを読み込むと重くなるため、この「軽量な一覧用コピー」を別途保持している設計になっている。

6.2 TOPとの同期（recordsが古くならないようにする仕組み）

sakaeIS_records_v1は次の3つの経路で更新される。

1. 作業票側のsyncToOrderRecords()：作業票を保存するたびに、共通項目（社内No・客先・型式・受注日・納期・**品名・数量**）をsakaeIS_records_v1の該当レコードへ直接反映する（同一ブラウザ内であれば即座に反映される、最も確実な経路）
2. TOP画面のstorageイベントリスナー（syncRecordFromBuhinOrder()）：別タブ・別PCで品番別データが変わった際、Realtime経由で届く変更をsakaeIS_records_v1へ反映する
3. 受注に関する連絡書の保存（syncToBuhinState()）：連絡書側で客先・品名・数量等を変更した場合、品番別データ（bs.order）へ直接書き込む

その上で、**TOP画面のカード一覧render()は、表示直前に品番別データのbs.orderを再マージしてから描画する**（mergedRecForForm()相当の処理をカード単位で適用）。これにより、上記1～3の同期が何らかの理由で一瞬遅れていても、カード一覧を開いた時点の実際の値が表示される。詳細モダールも同様にmergedRecForForm()で最新値を参照しており、sakaeIS_records_v1の古いキャッシュ値だけを表示することはない。

6.3 品番別データの構造（sakaeIS_buhinhyoMock_v1_<品番>）

```
{  
  order: {  
    productNo, // 品番（新規作成時の社内No.で決まる。作業票では読み取り専用）  
    orderNo,   // 社内No.  
    customer,  // 客先  
    model,     // 型式・型番  
    orderDate, // 受注日  
    dueDate,   // 納期  
    issueDate, // 発行日（受注日とは別項目。常に同期しない）  
    itemName,  // 品名
```

```

    qty,          // 数量
    // 以下、既存の作業票固有の管理項目（作成日・作成者・発送日・組付有無等）も保持
  },
  orderNotice: {
    // 受注に関する連絡書にしかない項目
    tCode, category, subject, shikyu, komeSu, amount, extraDocs,
    infoForecast, drafter, mfgResp, salesResp, designResp,
    issueDate1, issueDate2, note, status2,
    designSchedule, designPlan, costSummary, registration
  },
  arrangementCheck: {
    // 手配チェックシート固有の入力状態
    category,      // 区分（新作／修改／予備品）
    inputDate,     // 入力日
    items,         // 12項目分の { deadline, completedDate, checkedBy }
    approvedBy,    // 承認
    createdBy      // 作成
  },
  parts: [
    // 既存の部品・工程データ（変更なし）
  ]
}

```

上記は実装コードと照合済みの構造である。order・orderNotice・arrangementCheckは同一オブジェクト内の別プロパティであり、別々のlocalStorageキーには分かれていない（新しいキーを増やさず、既存の品番別データ内にまとめる方針で実装されている）。

6.3.1 共通項目（作業票と受注に関する連絡書で共有する項目）

order配下の項目（品番・社内No・客先・型式・受注日・納期・発行日・品名・数量）は、**作業票と受注に関する連絡書の両方が同じorderオブジェクトを直接読み書きする**。別領域（orderNotice側）へ複製することはしない。作業票で変更すれば連絡書にも反映され、連絡書で変更すれば作業票にも反映される（画面をまたいだ「同期」というより、同じデータを2つの画面が参照している）。

ただし、issueDate（発行日）とorderDate（受注日）は明確に別項目であり、一方を変更してももう一方は自動的に変わらない。

6.3.2 連絡書固有項目

orderNoticeには、受注に関する連絡書だけに存在する項目（品種・見積・登録事項・特記事項など）を保存する。作業票側には対応する入力欄が無い。

6.3.3 手配チェックシート固有項目

arrangementCheckには、手配チェックシートの入力状態（12項目の完了日・確認者、区分、入力日、承認者・作成者）を保存する。項目定義は現物の帳票「生産管理手配チェックシート」の実データと突き合わせて確認した12項目をそのまま採用している（詳細は手配チェックシート章参照）。

6.4 マイグレーション（互換処理）

品番別データを読み込むmigrateState()は、以下を補完する。

- orderNoticeが無ければ空オブジェクトを補完
- arrangementCheckが無ければ、12項目分の空データを補完
- order.issueDate／order.itemName／order.qtyが無ければ空文字列を補完
- 既存の部品・工程データ（parts[]）はそのまま保持し、破壊しない

重要: migrateState()はこれらを読み込み時にメモリ上のオブジェクトへ一時的に補完するだけであり、この関数自身がlocalStorageへの書き込み（自動保存）を行うことはない。実際にlocalStorageへ保存されるのは、ユーザーが何らかの入力を行い、明示的にsave()が呼ばれた時のみである。

upgradeDaysOnce() (工程の日数フィールドの一度限りの底上げ移行) は、Supabase 初回同期 (initialSync()) の完了を待たずに実行すると、直後の初回同期が移行前の古いリモート値でローカルの変更を上書きしてしまうレース条件が起こり得る。これは既存のclearStaleHoursOnce() (見積工数の一括クリア処理) が先に採用していた対策パターンで、window.sakaeSupabase (boot())完了後にセットされるグローバル変数) の存在を500ms間隔でポーリングし、同期完了 (または最大20秒のタイムアウト) を待ってから実行する。今回upgradeDaysOnce()もこのパターンに統一した。

6.5 localStorageキー一覧

キー (プレフィックス)	用途	同期範囲
sakaeIS_buhinhyoMock_v1_<品番>	作業票本体 (order+orderNotice+arrangementCheck+parts)。品番ごとに1レコード。 システム全体の唯一の基準データ	全員で共有
sakaeIS_records_v1	案件一覧 (TOP画面) の案件カードのインデックス	全員で共有
sakaeIS_customCodeColors_v1	工程コードごとの自動採番カラー	全員で共有
sakaeIS_kobetsuMock_confirm_v1	個別日程表・作業票が共有する「確定 (ロック)」状態	全員で共有
sakaeIS_kobetsuMock_checks_v1	個別日程表の担当者別進捗チェック (担当者名は編集可能)	ローカル運用の目印
sakaeIS_sagyohyoMock_memoVertical	作業票の注記欄・縦書き表示切り替え	全員で共有
sakaeIS_jissekiLog_v1	実績入力のログ (完了記録)	全員で共有
sakaeIS_zanpinMock_checks_v1	工程別残品表の「ローカル目印」チェック	ローカル運用の目印
sakaeIS_shikyuNouhinBoard_items / _view_v1	支給・納品一覧 (他の帳票と非連携の独立掲示板)	全員で共有
sakaeIS_daysUpgrade3_v1 / sakaeIS_hoursClear_v1	一度限りのデータ移行処理の実行済みフラグ	ローカル

その他、NC日程表・各工程の日程表のバー配置・付箋メモ等のキーは既存仕様のまま変更していない。

7 画面別技術仕様

7.1 案件一覧 (SAKAE SEIKI-iS.html)

- sakaeIS_records_v1を基に案件カードを描画する。表示直前に品番別データのbs.orderをマージするため、recordsの値が一瞬古くても正しい内容が表示される
- カード本体（または「作業票を開く／作成する」ボタン）をクリックすると、sakaeSyncFlush()を待ってから同一タブで作業票へ直接遷移する
 - 既に品番別データが存在する場合は「作業票を開く」
 - 存在しない場合は「作業票を作成する」
- カードの「詳細」ボタンをクリックすると、受注に関する連絡書の詳細モーダルを表示する(mergedRecForForm())で最新のbs.order／bs.orderNoticeをマージしてから表示)
- 新規登録（＋新規作成、社内No.の入力のみ）の直後は、受注に関する連絡書ではなく作業票へ直接遷移する
- 検索・ひな形（テンプレート）からの複製・JSON書き出し／読み込み・CSV書き出し等の既存機能は維持している
- ひな形からの複製では、部品・工程ルートは引き継ぐが、orderNotice（連絡書固有項目）・arrangementCheck（手配チェックシート固有項目）は引き継がない（新しい案件は完了状態・確認者等が空の状態から始まる）

7.2 作業票 (作業票_モック.html)

- 案件の中心画面。品番ごとの部品構成・工程ルートを編集するマスター画面であると同時に、関連画面への入口を兼ねる
- 画面上部に4つの関連機能ボタンを配置している

ボタン	内容
[印刷] 作業開始印刷セット	表紙・手配チェックシート・カンバンの入口画面を新しいタブで開く
[日程] 個別日程表を開く	同じ品番の個別日程表を新しいタブで開く
[書類] 受注に関する連絡書	同じ品番の連絡書を同一タブで開く
[印刷] 現行の作業票印刷	部品ごとの作業票印刷画面（既存機能）を新しいタブで開く

- 基本情報欄に発行日・品名・数量を追加した（従来は連絡書側にしか無かった項目を、作業票側でも編集できるようにした共通項目）
- 「工程と納期を確定する」チェックがsakaeIS_kobetsuMock_confirm_v1を介して個別日程表とロック状態を共有（片方で確定すると両方ロックされる）
- 共通項目（客先・型式・受注日・納期・品名・数量・社内No）はbs.orderへ直接保存し、TOPのsakaeIS_records_v1へもsyncToOrderRecords()で同期する
- 個別日程表・作業票印刷・作業開始印刷セットへの遷移は、いずれも遷移前に入力確定（blur()）・保存（save()）・sakaeSyncFlush()を行い、新しいタブは新規タブを開く処理とポップアップ対策で説明した3ステップ方式で開く
- ?productNo=パラメータが無い場合はseedData()によるサンプルデータにフォールバックする

7.3 個別日程表 (個別日程表_モック.html)

既存の双方向連携は変更していない。

- 作業票と同じ品番別データを参照し、双方向にリアルタイム同期する
- 作業票の日付変更でバーが更新され、バー移動で作業票の日付が更新される
- 「工程と納期を確定する」状態を作業票と共有する
- 作業票から「[日程] 個別日程表を開く」ボタンで直接開く
- 進捗チェック欄の担当者名は、以前は固定4名だったが、現在は画面上で追加・改名・削除できる

7.4 作業開始印刷セット（作業開始印刷セット_モック.html）

作業開始時に使用する3帳票（作業票表紙・手配チェックシート・カンバン）への入口専用画面。

- 対象品番・社内No・客先・品名を表示する（bs.orderから取得）
- 3帳票は用紙サイズ・向きが異なる（A5横／A4横／A4縦）ため、**1回の印刷ジョブでまとめて出力する設計にはしていない**。この画面から帳票ごとに「表示・印刷」ボタンで個別に新しいタブを開き、順番に印刷する
- 各帳票へのリンクは静的な
- 別タブ・別PCでの品番別データの変更をstorageイベントで購読し、表示中の対象品番情報を更新する

7.5 作業票表紙（作業票表紙_モック.html）

- 用紙：A5横（@page { size: A5 landscape; margin: 5mm; }）
- 印刷専用ビュー。共通項目の再入力できない（修正する場合は作業票へ戻って編集する）
- bs.orderから自動反映: 社内No・発行日・客先・品番・品名・型番・数量・受注日・納期
- 部品データ（parts[].processes[]）から実際に集計: 工程（使われている工程コードの一覧）・着手（最も早い日付）・完成（最も遅い日付）
- 実働Hは、実績入力のログから集計する（実績入力章参照）。**工程の予定工数・見積工数を実績として使用しない**
- 機械名・作業No・外注／担当者・見積Hは、現時点でシステム内に対応するデータ源が無いため空欄のまま印刷する（必要であれば手書きで記入する運用を想定）

7.6 手配チェックシート（手配チェックシート_モック.html）

- 用紙：A4横（@page { size: A4 landscape; margin: 7mm; }）
- 基本情報（社内No・品番・型番・品名）はbs.orderから表示（読み取り専用）
- 固有情報（区分・入力日・12項目の完了日／確認者・承認／作成）はbs.arrangementCheckへ保存する
- 12項目（現物の帳票「生産管理手配チェックシート」と一致させたもの）

項目	目安日数
設計図面	1日
材料手配（支給材料）	2日
材料手配（購入材料）	3日
日程表作成	2日
作業票作成	3日
出図	3日
支給品申請	5日
購入品手配	5日
加工外注手配	6日
材料入荷チェック	4日
購入品入荷チェック	10日
加工外注入荷チェック	－（期限計算なし）

- 営業日計算は「入力日から、土日だけを除外して指定日数を加算する」簡易計算（**祝日は考慮しない**）。現物Excelの目安日時・期限の実データと突き合わせて一致することを確認済み
- 別タブ・別PCの変更をstorageイベント（対象品番のキーのみ）で反映する。ただし入力中のフォーカス欄（INPUT/SELECT）がある場合は、Realtimeによる上書きを保留する
- 保存ボタンおよび印刷直前（beforeprint）に、その時点の入力内容をlocalStorageへ保存する

7.7 カンバン（カンバン_モック.html）

- 用紙：A4縦（@page { size: A4 portrait; margin: 6mm; }）

- 1ページ内に同一内容のカンバンを上下2面配置し、中央に切り取り線を表示する
- 印刷時、.sheet要素のwidth: 198mm; height: 285mm; (=A4からmarginを引いた印刷可能領域と一致する寸法) をautoにリセットしない。2つのパネルはflex: 1 1 50%でこの高さを正確に半分ずつ使うため、heightをautoにすると内容の少なさでページ下半分が空白になる（実機の印刷プレビューで確認された不具合であり、修正済み）
- 主表示は型番（42px）と品名（24px）。現物Excel「看板」シートが型番・品名の2行だけの簡素な作りだったため、これに合わせて大きく中央に表示している
- 社内No・数量・客先・納期・品番は、現物Excelには無い項目（作業指示書の指定により追加）で、主表示の邪魔にならないようパネル下端に小さく分離して表示する
- 基本情報はすべてbs.orderから表示する印刷専用ビュー

7.8 受注に関する連絡書

- 作業票上部の「[書類] 受注に関する連絡書」ボタンから、対象品番を指定して同一タブで開く（../SAKAE_SEIKI-iS.html?view=orderNotice&productNo=<品番>のディープリンク）
- 共通項目（客先・品名・数量・受注日・納期・型式・社内No）はbs.orderを直接編集する。別領域へ複製しない
- 固有項目（品種・見積・登録事項・特記事項等）はbs.orderNoticeに保存する
- 「← 作業票に戻る」ボタンで、対象品番を明示して作業票へ戻る（history.back()ではなく、sakaSyncFlush()後にlocation.hrefで明示的に戻る）
- TOPの詳細モーダルも含め、常にmergedRecForForm()で最新の品番別データを参照する
- 受注に関する連絡書を先に完成させなくても、作業票と個別日程表は作成できる

7.9 作業票印刷（作業票印刷_モック.html）

- 作業票のデータから部品単位の印刷用シートを生成する（既存機能、変更なし）
- subNo（枝番）でソート・フィルタ（自然数ソート）
- 枝番000（ASSY/検査工程扱い）を除く全部品を対象に、1部品=1枚（工程数に応じて複数枚）で出力

7.10 NC日程表（NC日程表_モック.html）

- 機種コード（ミ81／ミ51／ミ／タテ）ごとにシートが分かれる、機械×日付の2軸グリッド画面（既存機能、変更なし）
- 右クリックメニューからバーの設置・編集・割り当て解除・削除、および元の工程別日程表ページへの遷移ができる
- 自由記述バー（工程IDを持たないバー）は白／黄／緑の3色から選択可

7.11 実績入力（実績入力_モック.html）

- バーコード（Code39）読み取り、または品番一覧からの手動選択の2ルートで対象を特定し、完了記録をsakaIS_jissekiLog_v1へ追記する
- 記録項目: 対象品番（productNo）・日付・社員番号・段取（setupHours）・有人（mannedHours）・無人（unmannedHours）・夜間（nightHours）の各時間、数量、設備、その他備考、完了／未完
- 完了実績は、対象品番に一致するものだけを合計して**作業票表紙の実働H集計に使用する**（作業票表紙章参照）。実績が1件も無ければ実働Hは空欄になる。**予定工数・見積工数は実働Hとして使用しない**
- あくまで「ログの追記」であり、作業票のデータそのものは変更しない
- 実績ログのキー（sakaIS_jissekiLog_v1）は、作業票表紙側ではstorageイベントを購読していないため、別PCで実績を登録しても、既に開いている作業票表紙のタブには即時反映されない（次回そのタブを再読み込みした時点で反映される）

7.12 工程別残品表（残品表_モック.html）

- 「作業票の全工程－実績入力の完了ログ」を都度計算する派生ビュー（既存機能、変更なし）

7.13 支給・納品一覧（支給納品一覧_モック.html）

- 他の画面と一切データ連携しない、完全に独立した掲示板（既存機能、変更なし）

7.14 各工程の日程表（各工程の日程表/）

- _procGanttEngine.jsが全工程共通の描画・操作エンジン（既存機能、変更なし）

8 既知の制限・注意点

1. **機械名・作業No・外注／担当者・見積Hのデータ源が無い**: 作業票表紙にこれらの欄はあるが、システム内に対応する入力項目が無いため常に空欄になる。手書きでの追記を前提にしている
2. **手配チェックシートの営業日計算は祝日非対応**: 土日だけを除外する簡易計算であり、祝日・会社休日は考慮しない
3. **印刷3帳票は個別印刷**: 用紙サイズ・向きが異なるため、作業開始印刷セットの3帳票は1回の印刷ジョブではなく、帳票ごとに個別に印刷する設計
4. **新規タブを開く処理はポップアップ対策が必須**: 非同期のflush処理後にwindow.open()を呼ぶとポップアップブロックされるため、クリック直後に空タブを開いてからURLを設定する方式を採用している。新しくタブを開く機能を追加する場合はこのパターンに従うこと
5. **TOPのrecordsはインデックス**: sakaeIS_records_v1は表示用の軽量インデックスであり、表示時には必ず品番別データ (bs.order) を再マージしてから使う設計になっている。recordsの値を無条件に信頼しないこと
6. **品番 (productNo) は作業票側からは変更できない**: 品番は新規作成時の社内No.で決まり、作業票の品番欄は読み取り専用。社内No. (orderNo) 自体は受注に関する連絡書側で変更でき、その場合はmigrateProductData()が旧品番のデータを新品番へ引き継ぐ
7. **実績Hはリアルタイム更新されない**: 作業票表紙は品番別データの変更のみを購読しており、実績ログ (sakaeIS_jissekiLog_v1) の変更は購読していない。別PCで実績を登録しても、既に開いている作業票表紙タブには次回読み込みまで反映されない
8. **push未完了のままのページ遷移によるデータロス (一般論)**: 500msデバウンス以内に別ページへ遷移すると変更が失われる可能性があるという制約自体は残っている。上記の主要な遷移経路にはsakaeSyncFlush()による対策を入れているが、対策していない遷移経路 (例: ブラウザの戻る/進む操作やタブを閉じる操作) では引き続きこのリスクがある
9. **単一テーブルのKVモデルの限界**: 全データが1テーブルのJSONBカラムに入っているため、SQLレベルでの集計・部分更新・インデックス最適化ができない
10. **RLSがユーザー単位で分離されていない**: 認証済みユーザーは全キーを読み書きできる。社員間の権限分離は無い
11. **JSON入出力は全置換**: 案件一覧のJSON読み込みは現在の全データを置き換える (差分マージなし)。ただし品番別データ内のorderNotice・arrangementCheck・issueDate・itemName・qtyは、sakaeIS_buhinhy品番>キーごとまとめて書き出し・読み込みされるため、追加のコード対応なしでJSON入出力の対象に含まれている

8.1 動作確認済み仕様 (実機Chromeで確認)

以下は、実機のGitHub Pages版をChromeで開き、印刷プレビューまたはPDF保存で実際に確認済みの仕様である。

- 作業票表紙: A5横・1ページに収まる
- 手配チェックシート: A4横・1ページに収まる
- カンバン: A4縦・1ページ・上下2面に正しく収まる (切り取り線含む)

9 命名・コーディング規約（観察されたパターン）

- localStorageキーは必ずsakaeIS_プレフィックス（同期対象の判定に使われるため必須）
- バージョン管理のため_v1等のサフィックスを付ける慣習（スキーマ変更時は_v2にする、またはマイグレーションのような移行処理を書く）
- 動的に生成するDOM要素（コンテキストメニュー・ポップアップ・トースト等）は、対応する静的HTMLが用意されていないことが多く、各JSファイル内でdocument.getElementByIdが無ければcreateElementする自己完結パターンが多用されている
- 日本語のコメントで「なぜそうしているか」を明記するスタイルが徹底されている（実装の意図・過去に踏んだ落とし穴が書かれていることが多いので、変更前に必ず読むこと）
- 相対パスの解決に依存している箇所が多い（各工程の日程表/のように階層が変わるフォルダでは、_sakaeSync.js側で自分のscriptタグのsrcから相対パスを逆算する仕組みが使われている）
- 新しいタブを開くボタンは、原則として新規タブを開く処理とポップアップ対策の3ステップ方式（空タブを先に開く）を使うこと

10 デプロイ手順

1. SAKAE SEIKI-is/配下（作業用フォルダ）で編集
2. 変更したファイルをmikoshiya-seiki-is/配下の同一パスへコピー（diffで一致確認）
3. GitHub Desktopでコミット・プッシュ
4. GitHub Pagesが自動デプロイ（反映まで数十秒～数分。直後は旧アセットがブラウザ/CDNにキャッシュされている場合があるため、確認時はハード再読み込み推奨）
5. 新しいリポジトリを作る場合は、プッシュ前に.nojekyllを作成しておくこと（ディレクトリ構成章参照）

11 用語・旧名称対応

現在の正式名称	旧・略称（画面名としては使用しない）
案件一覧	受注連絡書一覧
受注に関する連絡書	受注連絡書 / 受注確認書 / 受注書 / 受注票

社外・テスト参加者向けの画面文言や一部変数名には、上記の旧名称が内部コード名（変数名・関数名・localStorageキーの一部）として残存している箇所がある。これはコード上のリネームであり、実際の会社名・ブランドは「SAKAE SEIKI-iS」（榮製機株式会社）。内部コード名や過去仕様の説明として必要な場合のみ、旧名称であることを明記した上で使用する。